

1. Match, cobertura y granularidad de bc3.pers

Este es el primer query. Es importante que usemos `COUNT(p.party_id)` y no `COUNT(*)`, porque estamos haciendo `LEFT JOIN`.

```
pers_match_granularity_qa <- sdf_sql(
  sc,
  "
  WITH cohort AS (

    SELECT
      party_id,
      observation_date,
      label_3m

    FROM bu_frd.model_features_ab_final_2022_2025
  ),

  x AS (

    SELECT
      f.party_id,
      f.observation_date,
      f.label_3m,

      COUNT(p.party_id) AS matched_rows,

      COUNT(DISTINCT p.birth_dte)
      AS distinct_birth_dates

    FROM cohort f

    LEFT JOIN bc3.pers p

      ON f.party_id = p.party_id
      AND p.data_date = f.observation_date
      AND p.data_date >= TO_DATE('2022-01-01')
      AND p.data_date < TO_DATE('2026-01-01')

    GROUP BY
      f.party_id,
      f.observation_date,
      f.label_3m
  )

  SELECT
```

```

COUNT(*) AS analytical_parties,

SUM(
  CASE WHEN matched_rows = 0
    THEN 1 ELSE 0 END
) AS parties_without_pers_record,

SUM(
  CASE WHEN matched_rows = 1
    THEN 1 ELSE 0 END
) AS parties_one_pers_record,

SUM(
  CASE WHEN matched_rows > 1
    THEN 1 ELSE 0 END
) AS parties_multiple_pers_rows,

SUM(
  CASE WHEN distinct_birth_dates = 0
    THEN 1 ELSE 0 END
) AS parties_without_birth_date,

SUM(
  CASE WHEN distinct_birth_dates > 1
    THEN 1 ELSE 0 END
) AS parties_multiple_birth_dates,

MAX(matched_rows)
  AS max_rows_same_party_date,

MAX(distinct_birth_dates)
  AS max_birth_dates_same_party

FROM x
"
) %>%
collect()

```

`pers_match_granularity_qa`

Idealmente queremos:

<code>analytical_parties</code>	688589
<code>parties_without_pers_record</code>	0
<code>parties_multiple_birth_dates</code>	0

No pasa nada si existen varias filas con **el mismo birth_dte**; eso se puede deduplicar. Lo problemático sería tener dos fechas de nacimiento diferentes para el mismo party en la misma **observation_date**.

2. Cobertura separada positives vs negatives

Esto es importante para el requirement de balance.

```
pers_coverage_by_label <- sdf_sql(
  sc,
  "
  WITH cohort AS (

    SELECT
      party_id,
      observation_date,
      label_3m

    FROM bu_frd.model_features_ab_final_2022_2025
  ),

  x AS (

    SELECT
      f.party_id,
      f.observation_date,
      f.label_3m,

      COUNT(p.party_id) AS matched_rows,
      COUNT(DISTINCT p.birth_dte) AS distinct_birth_dates

    FROM cohort f

    LEFT JOIN bc3.pers p

      ON f.party_id = p.party_id
      AND p.data_date = f.observation_date

    GROUP BY
      f.party_id,
      f.observation_date,
      f.label_3m
  )

  SELECT
```

```

label_3m,

COUNT(*) AS parties,

SUM(
    CASE WHEN matched_rows > 0
        THEN 1 ELSE 0 END
) AS parties_with_pers_record,

SUM(
    CASE WHEN distinct_birth_dates = 1
        THEN 1 ELSE 0 END
) AS parties_with_birth_date,

SUM(
    CASE WHEN distinct_birth_dates = 0
        THEN 1 ELSE 0 END
) AS parties_without_birth_date,

SUM(
    CASE WHEN distinct_birth_dates > 1
        THEN 1 ELSE 0 END
) AS parties_birth_date_conflict,

SUM(
    CASE WHEN distinct_birth_dates = 1
        THEN 1 ELSE 0 END
) * 1.0 / COUNT(*) AS birth_date_coverage

FROM x

GROUP BY label_3m

ORDER BY label_3m
"
) %>%
collect()

pers_coverage_by_label

```

Esto nos permite saber si `birth_dte` está igualmente disponible para positivos y negativos.

3. Solo si aparecen múltiples fechas de nacimiento

Si en el query 1:

```
parties_multiple_birth_dates > 0
```

ejecutamos esto:

```
pers_birth_conflicts <- sdf_sql(  
  sc,  
  "  
  WITH cohort AS (  
  
    SELECT  
      party_id,  
      observation_date,  
      label_3m  
  
    FROM bu_frd.model_features_ab_final_2022_2025  
  )  
  
  SELECT  
    f.party_id,  
    f.observation_date,  
    f.label_3m,  
  
    COUNT(*) AS rows_same_date,  
    COUNT(DISTINCT p.birth_dte) AS distinct_birth_dates,  
  
    COLLECT_SET(  
      CAST(p.birth_dte AS STRING)  
    ) AS birth_dates  
  
  FROM cohort f  
  
  INNER JOIN bc3.pers p  
  
    ON f.party_id = p.party_id  
    AND p.data_date = f.observation_date  
  
  GROUP BY  
    f.party_id,  
    f.observation_date,  
    f.label_3m  
  
  HAVING COUNT(DISTINCT p.birth_dte) > 1  
  
  LIMIT 100  
  "  
) %>%
```

```
collect()
```

```
pers_birth_conflicts
```

Si el resultado es cero, seguimos.

4. Crear una tabla pequeña para el EDA de edad

Una vez que los QA anteriores estén bien, recomiendo **materializar solo los 688,589 parties del análisis**, no `bc3.pers` completa.

Además, esta versión no selecciona arbitrariamente una fecha si hubiera conflicto.

```
sdf_sql(
  sc,
  "
CREATE TABLE bu_frd.eda_age_snapshot_2022_2025
STORED AS PARQUET
AS

WITH x AS (

  SELECT
    f.party_id,
    f.observation_date,
    f.observation_year,
    f.label_3m,

    COUNT(p.party_id) AS pers_rows,

    COUNT(DISTINCT p.birth_dte)
      AS distinct_birth_dates,

    CASE
      WHEN COUNT(DISTINCT p.birth_dte) = 1
      THEN MAX(p.birth_dte)
    END AS birth_dte

  FROM bu_frd.model_features_ab_final_2022_2025 f

  LEFT JOIN bc3.pers p

    ON f.party_id = p.party_id
```

```

        AND p.data_date = f.observation_date

    GROUP BY
        f.party_id,
        f.observation_date,
        f.observation_year,
        f.label_3m
    )

    SELECT
        *,

        CASE
            WHEN birth_dte IS NOT NULL
                AND birth_dte <= observation_date

            THEN CAST(
                FLOOR(
                    MONTHS_BETWEEN(
                        observation_date,
                        birth_dte
                    ) / 12
                )
                AS INT
            )
            END AS age_at_observation

    FROM x
    "
    )

```

Así:

$\lfloor \text{Age} = \left\lfloor \frac{\text{months_between}(\text{observation_date}, \text{birth_dte})}{12} \right\rfloor \rfloor$

La edad queda calculada al momento de observación, no a 2026.

5. QA de integridad de la tabla creada

```

age_table_integrity_qa <- sdf_sql(
  sc,
  "
  SELECT
    COUNT(*) AS total_rows,

```

```

COUNT(DISTINCT party_id)
  AS distinct_parties,

SUM(
  CASE WHEN label_3m = 1
    THEN 1 ELSE 0 END
) AS positives,

SUM(
  CASE WHEN label_3m = 0
    THEN 1 ELSE 0 END
) AS negatives

FROM bu_frd.eda_age_snapshot_2022_2025
"
) %>%
collect()

age_table_integrity_qa

```

Queremos exactamente:

total_rows	688589
distinct_parties	688589
positives	62599
negatives	625990

6. Data quality de **birth_dte** y edad

Este es importante antes de hacer gráficas.

```

age_quality_qa <- sdf_sql(
  sc,
  "
SELECT
  COUNT(*) AS total_parties,

SUM(
  CASE WHEN birth_dte IS NULL
    THEN 1 ELSE 0 END
) AS missing_birth_date,

SUM(
  CASE WHEN birth_dte > observation_date
    THEN 1 ELSE 0 END

```

```

) AS birth_after_observation,

SUM(
  CASE WHEN age_at_observation < 0
    THEN 1 ELSE 0 END
) AS negative_age,

SUM(
  CASE WHEN age_at_observation < 18
    THEN 1 ELSE 0 END
) AS age_under_18,

SUM(
  CASE WHEN age_at_observation > 100
    THEN 1 ELSE 0 END
) AS age_over_100,

MIN(age_at_observation)
  AS min_age,

MAX(age_at_observation)
  AS max_age

FROM bu_frd.eda_age_snapshot_2022_2025
"
) %>%
collect()

age_quality_qa

```

Importante: <18 o >100 son inicialmente **review flags**, no registros que yo borraría automáticamente.

7. Buscar valores sospechosos de birth_dte

Esto sirve para detectar defaults como 1900-01-01, fechas repetidas artificialmente, etc.

```

birth_date_frequency <- sdf_sql(
  sc,
  "
SELECT
  birth_dte,

```

```

COUNT(*) AS parties

FROM bu_frd.eda_age_snapshot_2022_2025

WHERE birth_dte IS NOT NULL

GROUP BY birth_dte

ORDER BY parties DESC

LIMIT 30
"
) %>%
collect()

birth_date_frequency

```

No significa que una fecha frecuente sea necesariamente inválida. Solo la revisamos.

8. Distribución continua de edad por label

Este será uno de los principales resultados descriptivos.

```

age_summary_by_label <- sdf_sql(
  sc,
  "
SELECT
  label_3m,

  COUNT(*) AS parties,

  COUNT(age_at_observation)
  AS parties_with_valid_age,

  AVG(age_at_observation)
  AS avg_age,

  STDDEV_SAMP(age_at_observation)
  AS std_age,

  MIN(age_at_observation)
  AS min_age,

```

```

PERCENTILE_APPROX(age_at_observation, 0.01)
  AS p01_age,

PERCENTILE_APPROX(age_at_observation, 0.05)
  AS p05_age,

PERCENTILE_APPROX(age_at_observation, 0.25)
  AS p25_age,

PERCENTILE_APPROX(age_at_observation, 0.50)
  AS median_age,

PERCENTILE_APPROX(age_at_observation, 0.75)
  AS p75_age,

PERCENTILE_APPROX(age_at_observation, 0.95)
  AS p95_age,

PERCENTILE_APPROX(age_at_observation, 0.99)
  AS p99_age,

MAX(age_at_observation)
  AS max_age

FROM bu_frd.eda_age_snapshot_2022_2025

WHERE age_at_observation IS NOT NULL

GROUP BY label_3m

ORDER BY label_3m
"
) %>%
collect()

```

age_summary_by_label

Esto nos permitirá comparar:

\[Age\mid label=1 \]

contra

\[Age\mid label=0 \]

sin depender solo de bandas arbitrarias.

9. Crear age bands y revisar balance

Usaría estas bandas:

<18
18-24
25-34
35-44
45-54
55-64
65-74
75+
MISSING/INVALID

El query:

```
age_band_balance <- sdf_sql(  
  sc,  
  "  
  WITH x AS (  
  
    SELECT  
      party_id,  
      label_3m,  
  
    CASE  
      WHEN age_at_observation IS NULL  
        THEN 'MISSING_INVALID'  
  
      WHEN age_at_observation < 18  
        THEN '<18'  
  
      WHEN age_at_observation BETWEEN 18 AND 24  
        THEN '18-24'  
  
      WHEN age_at_observation BETWEEN 25 AND 34  
        THEN '25-34'  
  
      WHEN age_at_observation BETWEEN 35 AND 44  
        THEN '35-44'  
  
      WHEN age_at_observation BETWEEN 45 AND 54  
        THEN '45-54'  
  
      WHEN age_at_observation BETWEEN 55 AND 64  
        THEN '55-64'
```

```
    WHEN age_at_observation BETWEEN 65 AND 74
      THEN '65-74'
```

```
    ELSE '75+'
  END AS age_band
```

```
FROM bu_frd.eda_age_snapshot_2022_2025
),
```

```
counts AS (
```

```
  SELECT
    age_band,
```

```
    SUM(
      CASE WHEN label_3m = 1
        THEN 1 ELSE 0 END
    ) AS positive_parties,
```

```
    SUM(
      CASE WHEN label_3m = 0
        THEN 1 ELSE 0 END
    ) AS negative_parties,
```

```
    COUNT(*) AS total_parties
```

```
  FROM x
```

```
  GROUP BY age_band
),
```

```
totals AS (
```

```
  SELECT
    SUM(positive_parties) AS total_positive,
    SUM(negative_parties) AS total_negative
```

```
  FROM counts
)
```

```
SELECT
  c.age_band,

  c.positive_parties,
  c.negative_parties,
  c.total_parties,

  c.positive_parties * 1.0 /
```

```

        t.total_positive AS pct_positive_distribution,

        c.negative_parties * 1.0 /
        t.total_negative AS pct_negative_distribution,

        CASE
            WHEN c.negative_parties > 0
            THEN
                (c.positive_parties * 1.0 / t.total_positive)
                /
                (c.negative_parties * 1.0 / t.total_negative)
            END AS representation_ratio,

        c.positive_parties * 1.0 /
        c.total_parties AS positive_rate_in_sample

FROM counts c

CROSS JOIN totals t

ORDER BY
    CASE c.age_band
        WHEN '<18' THEN 1
        WHEN '18-24' THEN 2
        WHEN '25-34' THEN 3
        WHEN '35-44' THEN 4
        WHEN '45-54' THEN 5
        WHEN '55-64' THEN 6
        WHEN '65-74' THEN 7
        WHEN '75+' THEN 8
        ELSE 9
    END
"
) %>%
collect()

age_band_balance

```

Esto será equivalente al análisis que acabamos de hacer por estado.

10. Medida formal de balance: Standardized Mean Difference

Además de la tabla descriptiva, recomiendo calcular esto.

$$SMD = \frac{\bar{x}_{positive} - \bar{x}_{negative}}{\sqrt{\frac{s_{positive}^2 + s_{negative}^2}{2}}}$$

Es una forma mucho más defendible de documentar si existe diferencia en edad entre los dos grupos.

```
age_smd <- sdf_sql(
  sc,
  "
  WITH stats AS (

    SELECT
      label_3m,

      AVG(age_at_observation)
      AS mean_age,

      VAR_SAMP(age_at_observation)
      AS var_age

    FROM bu_frd.eda_age_snapshot_2022_2025

    WHERE age_at_observation IS NOT NULL
      AND age_at_observation >= 0

    GROUP BY label_3m
  ),

  p AS (

    SELECT
      mean_age AS mean_positive,
      var_age AS var_positive

    FROM stats

    WHERE label_3m = 1
  ),

  n AS (

    SELECT
      mean_age AS mean_negative,
      var_age AS var_negative
```

```

FROM stats

WHERE label_3m = 0
)

SELECT
  mean_positive,
  mean_negative,

  mean_positive - mean_negative
    AS mean_difference,

  SQRT(
    (var_positive + var_negative) / 2.0
  ) AS pooled_sd,

  (
    mean_positive - mean_negative
  )
  /
  SQRT(
    (var_positive + var_negative) / 2.0
  ) AS standardized_mean_difference

FROM p
CROSS JOIN n
"
) %>%
collect()

age_smd

```

Para nuestro documento, no usaría un cutoff mecánico como “pass/fail”. Lo interpretaríamos junto con las distribuciones.

11. Estabilidad de edad por año

También lo recomiendo porque nuestro estudio cubre 2022–2025.

```

age_by_year_label <- sdf_sql(
  sc,
  "
SELECT
  observation_year,

```

```

label_3m,

COUNT(*) AS parties,

AVG(age_at_observation)
  AS avg_age,

PERCENTILE_APPROX(
  age_at_observation,
  0.50
) AS median_age,

PERCENTILE_APPROX(
  age_at_observation,
  0.25
) AS p25_age,

PERCENTILE_APPROX(
  age_at_observation,
  0.75
) AS p75_age

FROM bu_frd.eda_age_snapshot_2022_2025

WHERE age_at_observation IS NOT NULL

GROUP BY
  observation_year,
  label_3m

ORDER BY
  observation_year,
  label_3m
"
) %>%
collect()

age_by_year_label

```

Nos permite detectar, por ejemplo, si positivos de 2024 fueran considerablemente más jóvenes o mayores que los de 2022.

12. SMD por año

Como validación adicional:

```
age_smd_by_year <- sdf_sql(  
  sc,  
  "  
  WITH stats AS (  
  
    SELECT  
      observation_year,  
      label_3m,  
  
      AVG(age_at_observation)  
        AS mean_age,  
  
      VAR_SAMP(age_at_observation)  
        AS var_age  
  
    FROM bu_frd.eda_age_snapshot_2022_2025  
  
    WHERE age_at_observation IS NOT NULL  
  
    GROUP BY  
      observation_year,  
      label_3m  
  )  
  
  SELECT  
    p.observation_year,  
  
    p.mean_age AS positive_mean_age,  
    n.mean_age AS negative_mean_age,  
  
    p.mean_age - n.mean_age  
      AS mean_difference,  
  
    (  
      p.mean_age - n.mean_age  
    )  
    /  
    SQRT(  
      (p.var_age + n.var_age) / 2.0  
    ) AS standardized_mean_difference  
  
  FROM stats p  
  
  INNER JOIN stats n  
  
    ON p.observation_year = n.observation_year
```

```
AND p.label_3m = 1
AND n.label_3m = 0

ORDER BY p.observation_year
"
) %>%
collect()

age_smd_by_year
```

Qué necesito que me pases

No necesitas mandarme todos los outputs inmediatamente. Para avanzar de manera ordenada, primero pásame los resultados de:

1. `pers_match_granularity_qa`
2. `pers_coverage_by_label`
3. `age_quality_qa`
4. `age_summary_by_label`
5. `age_band_balance`
6. `age_smd`

Con esos seis podemos determinar si Age queda completamente validado.

Después usamos `age_by_year_label` y `age_smd_by_year` como **stability / appendix validation**.

Y una vez cerrado Age, nos moveríamos a **Credit Age**, que es el requisito adicional más delicado de la documentación porque tendremos que justificar exactamente qué fecha de `trades` representa el inicio del historial crediticio.